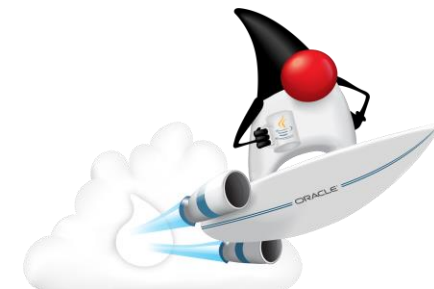


Programação Orientada a Objetos

- ✓ SOBRECARGA DE MÉTODOS
- ✓ ESTRUTURAS DE REPETIÇÃO
- ✓ ENCAPSULAMENTO DE DADOS
- ✓ MODIFICADORES DE ACESSO



Desafio

1- Faça um programa que solicite ao usuário um número, calcule e mostre a tabuada desse número.

```
import java.util.Scanner;
public class Tabuada {
    public static void main(String[] args) {
        // TODO code application logic here
        Scanner sc = new Scanner(System.in);
        System.out.println("Entre com um número inteiro: ");
        int x = sc.nextInt();
        System.out.println(x + " x 0 = " + x * 0);
        System.out.println(x + " x 1 = " + x * 1);
        System.out.println(x + " x 2 = " + x * 2);
        System.out.println(x + " x 3 = " + x * 3);
        System.out.println(x + " x 4 = " + x * 4);
        System.out.println(x + " x 5 = " + x * 5);
        System.out.println(x + " x 6 = " + x * 6);
        System.out.println(x + " x 7 = " + x * 7);
        System.out.println(x + " x 8 = " + x * 8);
        System.out.println(x + " x 9 = " + x * 9);
        System.out.println(x + " x 10 = " + x * 10);
    }
}
```



Estruturas de repetição

- ✓ Também conhecidas como laços (loop).
- ✓ Utilizadas para executar repetidamente uma instrução ou um bloco de instruções enquanto uma determinada condição for verdadeira.
- ✓ As estruturas de repetição em Java são:
 - `for`
 - `while`
 - `do ... while`



Estruturas de repetição

Para determinarmos qual é a estrutura mais adequada, devemos saber:

- ✓ o número de vezes que o trecho programa vai ser executado: laços contados
- ✓ ou a condição para que ela aconteça: laços condicionais

Laços contados: um **contador** irá auxiliar no laço. Neste laço, a repetição da estrutura repete-se até que o contador atinja o limite estipulado na condição.

Laços condicionais: o valor é desconhecido e devemos utilizar uma variável com valor predefinido em uma condição dentro do laço para finalizarmos a repetição.



Estrutura de repetição

Independente do tipo de laço, todos são constituídos de três partes:

- ✓ Inicialização(ões) da(s) variável(is) de controle
- ✓ Condição(ões)
- ✓ Atualização da(s) variável(is) de controle



Estrutura de repetição: for

```
for (início; condição; incremento/decremento)
{
    instruções;
}
```

Exemplo:

```
...
int n;
for (n=0; n < 10; n++) {
    JOptionPane.showMessageDialog(null, n);
}
...
```

Executa as <instruções> enquanto a <condição> for verdadeira, tendo no lugar de <início> uma variável de controle e em <incremento/ decremento> a alteração do valor desta variável.

Exemplo

```
import java.util.Scanner;

public class RepeticaoFor {
    public static void main(String[] args) {

        Scanner leitura = new Scanner(System.in);
        int soma = 0, cont, A;
        for (cont=0; cont<2; cont++) {

            A = leitura.nextInt();

            soma += A;

        }

        System.out.println("A média é: " +(soma/cont));

    }
}
```

Permite a leitura de entrada de dados no prompt

Condição de execução

Variável Contador

Variável Acumulador

Lê o que foi digitado como valor inteiro

Desafio

1- Faça um programa que solicite ao usuário um número, calcule e mostre a tabuada desse número.

```
import java.util.Scanner;
public class Tabuada {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.println("Entre com um número inteiro: ");
        int x = sc.nextInt();
        for(int i = 0; i <= 10; i++){
            System.out.println(x + " x " + i + " = " + x * i);
        }
    }
}
```


Estrutura de repetição: while

```
while (condição)
{
    instruções;
}
```

Exemplo:

```
...
int n = 1;
while (n <= 10) {
    n = n * 2; // é o mesmo que n *= 2;
}
JOptionPane.showMessageDialog(null,
"Resultado: " + n);
...
```

Executa as <instruções> enquanto a <condição> for verdadeira, quando a condição deixar de ser verdadeira ele para de executar as instruções.

Exemplo

```
import java.util.Scanner;
```

```
public class RepeticaoWhile {  
    public static void main(String[] args) {  
        Scanner leitura = new Scanner(System.in);  
        int A = 1, soma = 0, cont = 0;  
        while (A > 0) {  
            A = leitura.nextInt();  
            if (A > 0) {  
                soma = soma + A;  
                cont++;  
            }  
        }  
        System.out.println("A média é: " + (soma/cont));  
    }  
}
```

Variável
Contador

Condição de
execução

Variável
Acumulador

Estrutura de repetição: do-while

```
do {  
    instruções;  
}  
while (condição);
```

Exemplo:

```
...  
int n = 1;  
do {  
    n = n * 2; // é o mesmo que n *= 2;  
} while (n <= 10);  
    JOptionPane.showMessageDialog(null,  
    "Resultado: " + n);  
...
```

Executa as <instruções> uma vez sem verificar se a <condição> é verdadeira, no final desta execução verifica a <condição> e se ela for verdadeira, executa as instruções novamente, quando a condição deixar de ser verdadeira ele para de executar as instruções.

Exemplo

```
import java.util.Scanner;

public class RepeticaoDoWhile {
    public static void main(String[] args) {
        Scanner leitura = new Scanner(System.in);
        float A, soma = 0;
        String resp;
        int cont = 0;
        do {
            System.out.println("Informe um número");
            A = leitura.nextFloat();
            if (A > 0) {
                soma = soma + A;
                cont++;
            }
            System.out.println("Deseja continuar (S ou N)?");
            resp = leitura.next();
        } while (resp.equalsIgnoreCase("s"));
        System.out.println("A média é: " + (soma/cont));
    }
}
```

Lê o que foi
digitado como valor
float

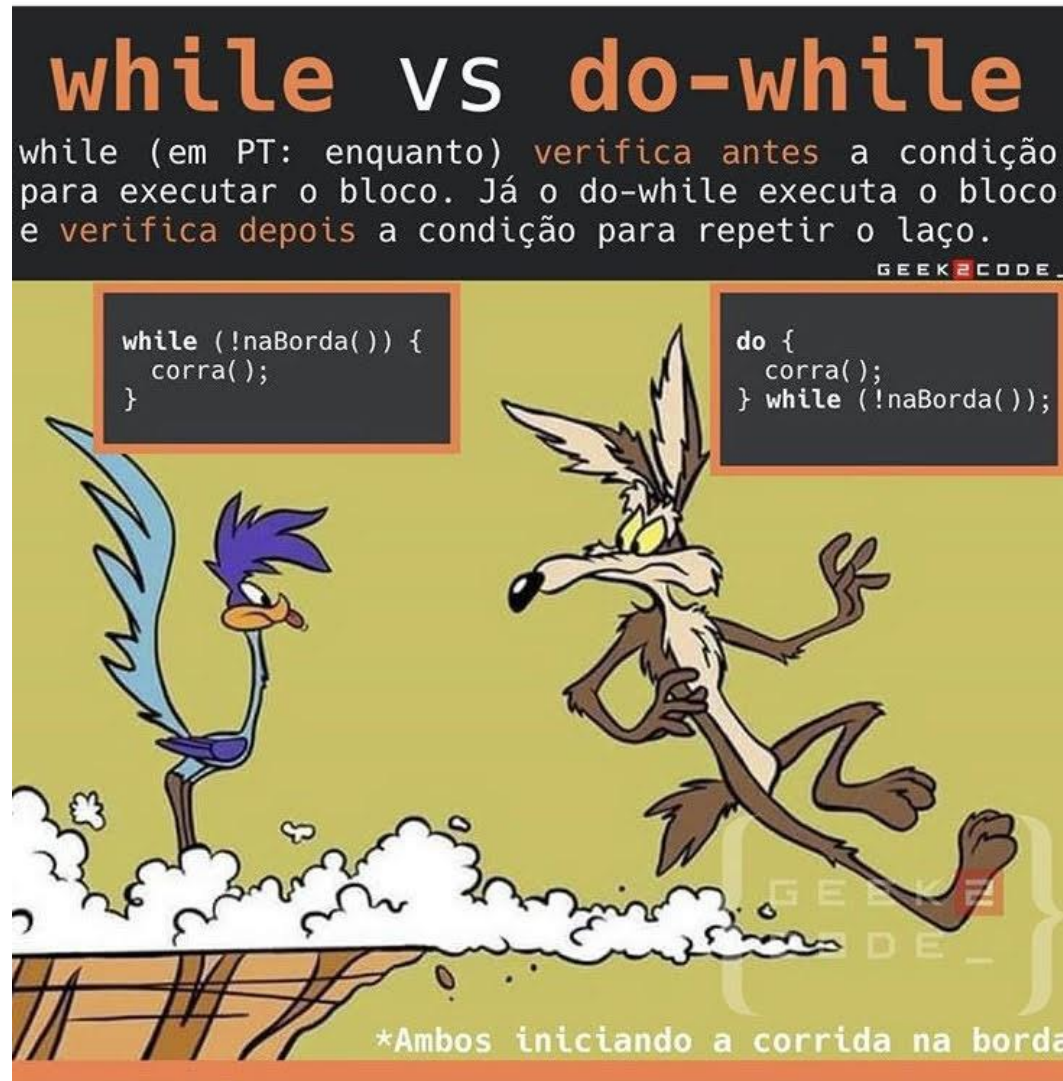
Variável
Contador

Lê a resposta que
define se a
repetição continua

Variável
Acumulador

Condição de execução

while vs do-while



Outros exemplos: estrutura for

```
import javax.swing.JOptionPane;

public class Exemplo1 {
    public static void main(String[] args) {
        int cont;
        String conteudo="";
        for (cont = 0; cont < 10 ; cont++)
        {
            conteudo = conteudo + "Número: " + cont + "\n";
        }
        JOptionPane.showMessageDialog(null, conteudo);
    }
}
```

Outros exemplos: estrutura while

```
import javax.swing.JOptionPane;
public class Exemplo4 {
    public static void main(String[] args) {
        int cont = 0;
        String conteudo="";
        while (cont < 10)
        {
            conteudo += "Número: " + cont + "\n";
            cont = cont + 1;
        }
        JOptionPane.showMessageDialog(null, conteudo);
    }
}
```

Outros exemplos: estrutura do-while

```
import javax.swing.JOptionPane;
public class Exemplo6 {
    public static void main(String[] args) {
        int cont=0;
        String conteudo="";
        do
        {
            conteudo += "Número: " + cont + "\n";
            cont = cont + 1; //ou cont++;
        } while (cont < 10);
        JOptionPane.showMessageDialog(null,conteudo);
    }
}
```


Desafio

2- Adivinhe meu número: Crie um jogo onde o computador escolhe um número inteiro aleatório entre 0 e 20.

- ✓ Leia a entrada do usuário para tentar acertar o número;
- ✓ Se errar informar ao usuário se o número é maior ou menor;
- ✓ Repetir até o usuário acertar.

Observação: Para gerar um número aleatório utilize a classe Random:

```
Random aleatorio = new Random();  
int num = aleatorio.nextInt(20);
```



Desafio

Desafio 2: Adivinhe o número

```
import java.util.Scanner;
import java.util.Random;
class Main {
    public static void main(String[] args) {
        //instância um objeto da classe Random usando o construtor básico
        Random gerador = new Random();
        int num = gerador.nextInt(20);
        int x, i = 0;
        Scanner sc = new Scanner(System.in);
        do{
            i++;
            System.out.println("Digite um número entre 0 e 20:");
            x = sc.nextInt();
            if(x == num){
                System.out.println("Parabéns, você acertou em " + i + " tentativas" );
            } else if (x < num) {
                System.out.println("O número pensado é maior");
            } else {
                System.out.println("O número pensado é menor");
            }
        } while (x != num);
    }
}
```



Encapsulamento de Dados

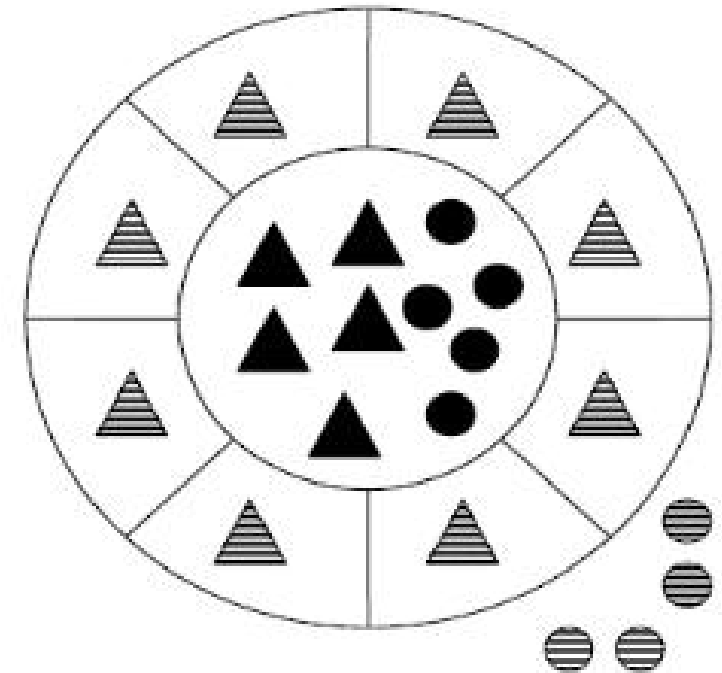
- ✔ O encapsulamento é as vezes referido como **ocultamento de informações**. Os usuários dos objetos não conhecem sua constituição e os utilizam através dos métodos públicos.
- ✔ O encapsulamento elimina dependências diretas na implementação, possibilitando a mudanças sem afetar outros sistemas que utilizem o objeto, desde que as assinaturas dos métodos não sejam alteradas.



Encapsulamento de Dados

public Menos restrito
↓
private Mais restrito

- CLASSE**
- ▲ métodos públicos
 - ▲ métodos privados
 - dados privados
 - dados públicos (não recomendável)



Encapsulamento de Dados

- ✓ Podemos bloquear o acesso aos atributos da classe.
- ✓ Criamos métodos (*set* e *get*) para acessar os atributos, já que eles não são mais acessíveis diretamente.
- ✓ Nesses métodos, podemos "validar" o que estão tentando armazenar nos atributos.

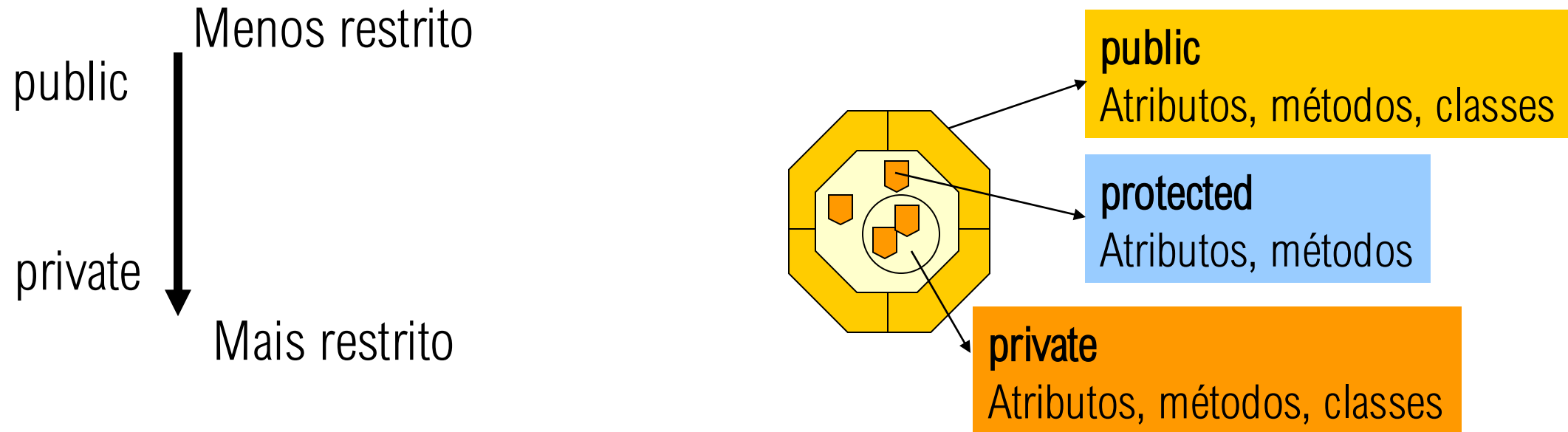


SEGURANÇA



Encapsulamento de Dados

É uma proteção adicional dos dados do objeto de possíveis modificações impróprias, forçando o acesso a um nível mais baixo para tratamento de dados.



Exemplo:

Quando temos um arquivo protegido por senha de acesso, podemos dizer que ele está protegido, pois, apenas podemos lê-lo sem fazermos alteração

Modificadores de acesso

- ✓ Há três **modificadores** de acesso: **public**, **protected** e **private**;
- ✓ Atributos e Métodos podem ter os **três níveis** de acesso.
- ✓ Elementos **públicos** podem ser acessados diretamente por qualquer outra classe, utilizando um ponto (.) após o nome da variável.
- ✓ Elementos **privados** e **protegidos** não podem ser acessados diretamente utilizando o ponto.



Modificadores de acesso

- ✓ **Private:** O modificador de acesso **private** é o mais restritivo de todos, variáveis e métodos com esse modificador são visíveis somente dentro da definição da própria classe, acessando-o diretamente ou através de uma instância da mesma classe.
- ✓ **Protected:** O modificador de acesso **protected** define que variáveis e métodos com esse modificador podem somente ser acessados por subclasses.
- ✓ **Public:** O mais abrangente de todos os tipos de acesso, o modificador **public** declara que elementos com esse modificador são acessíveis de qualquer classe Java.
- ✓ **Default:** Define que variáveis ou métodos podem somente ser acessados por classes do mesmo pacote.



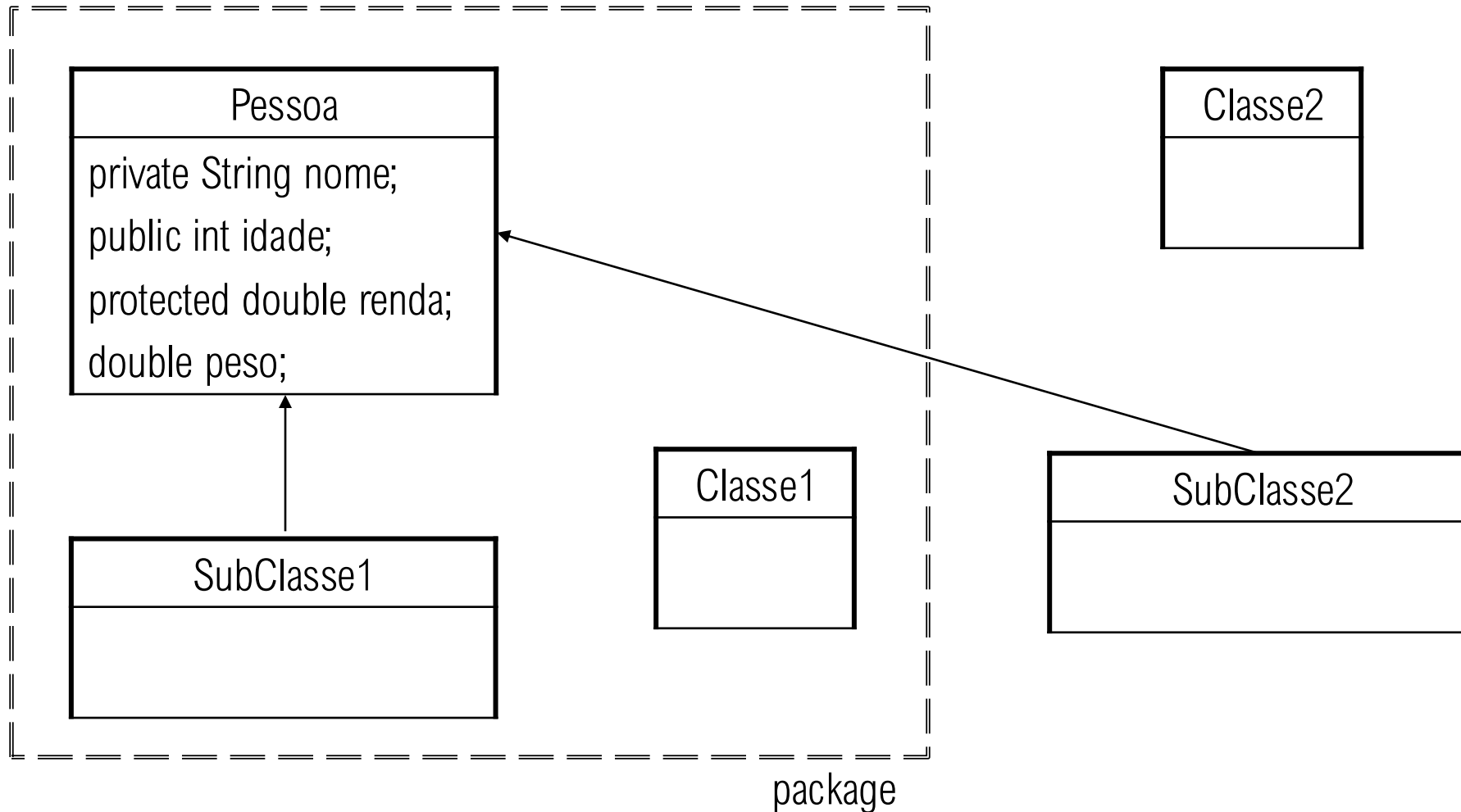
Modificadores de acesso

public	✓ Pode ser acessado por qualquer classe
protected	✓ Pode ser acessado por qualquer sub-classe ✓ Pode ser acessado por qualquer classe do mesmo pacote ✓ Pode ser acessado pela própria classe
default	✓ Pode ser acessado por qualquer classe do mesmo pacote ✓ Pode ser acessado pela própria classe
private	✓ Pode ser acessado somente pela própria classe



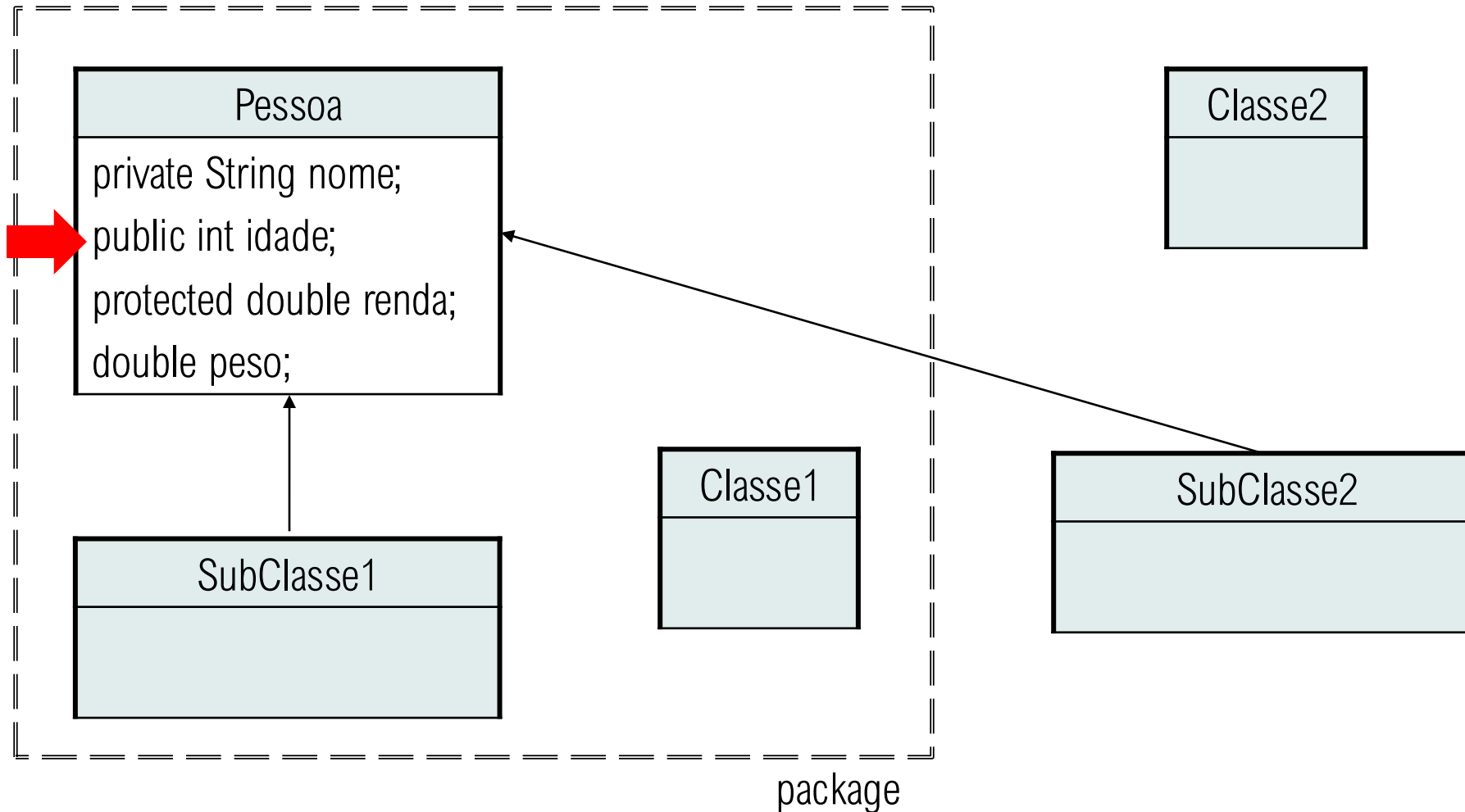
Modificadores de acesso

Visibilidade de variáveis e método: **Exemplo**



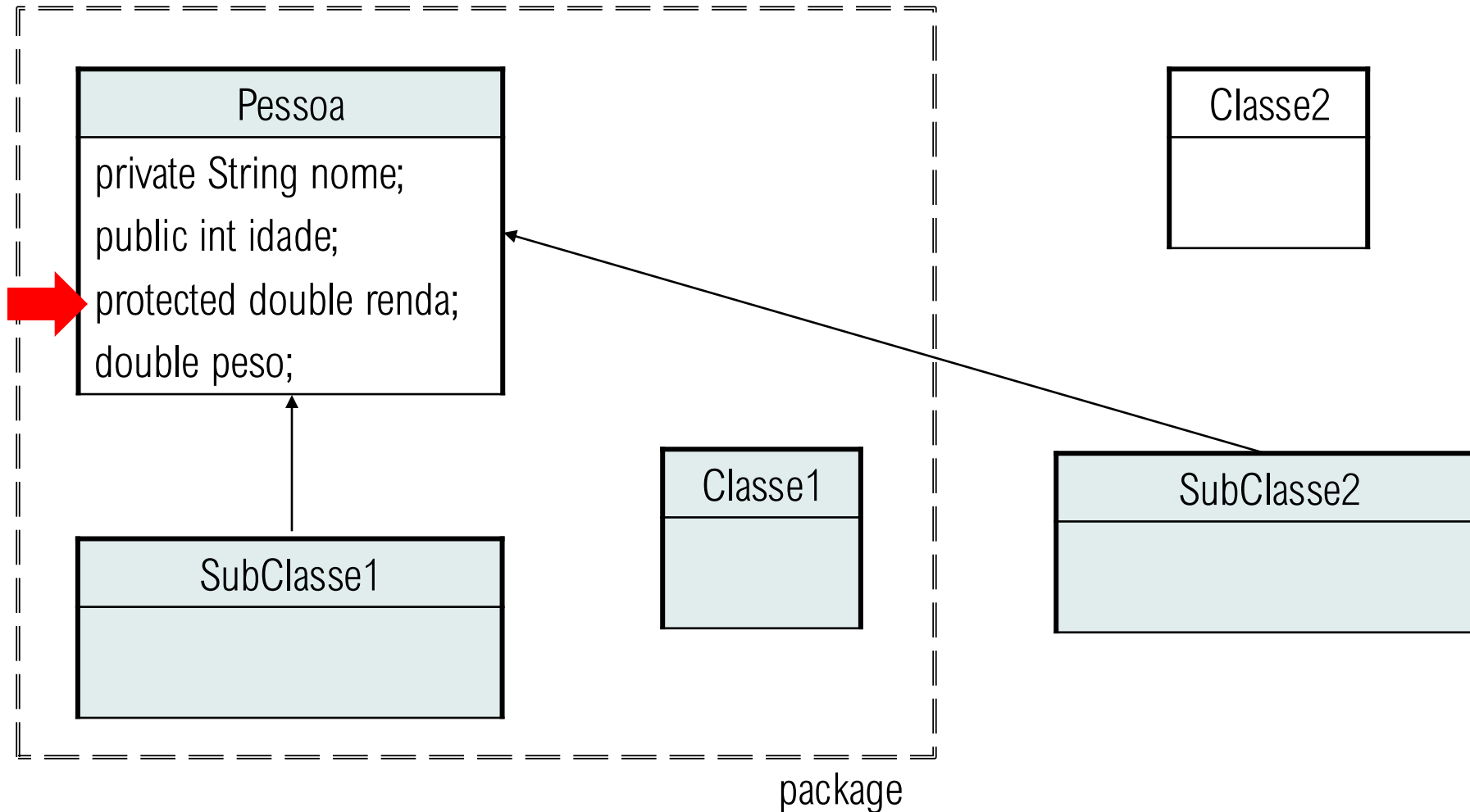
Modificadores de acesso

Visibilidade de variáveis e métodos com o modificador **public**.



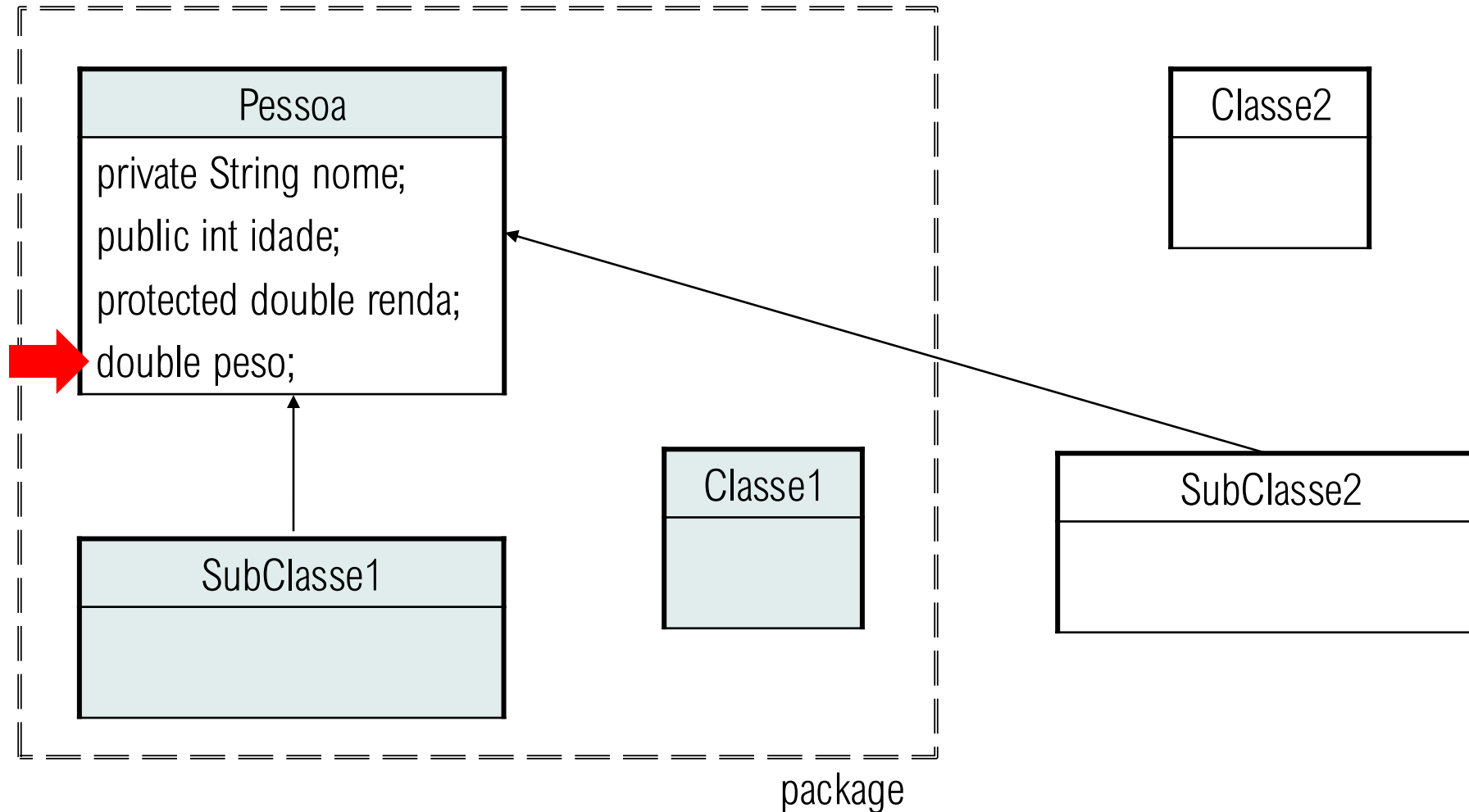
Modificadores de acesso

Visibilidade de variáveis e métodos com o modificador **protected**.



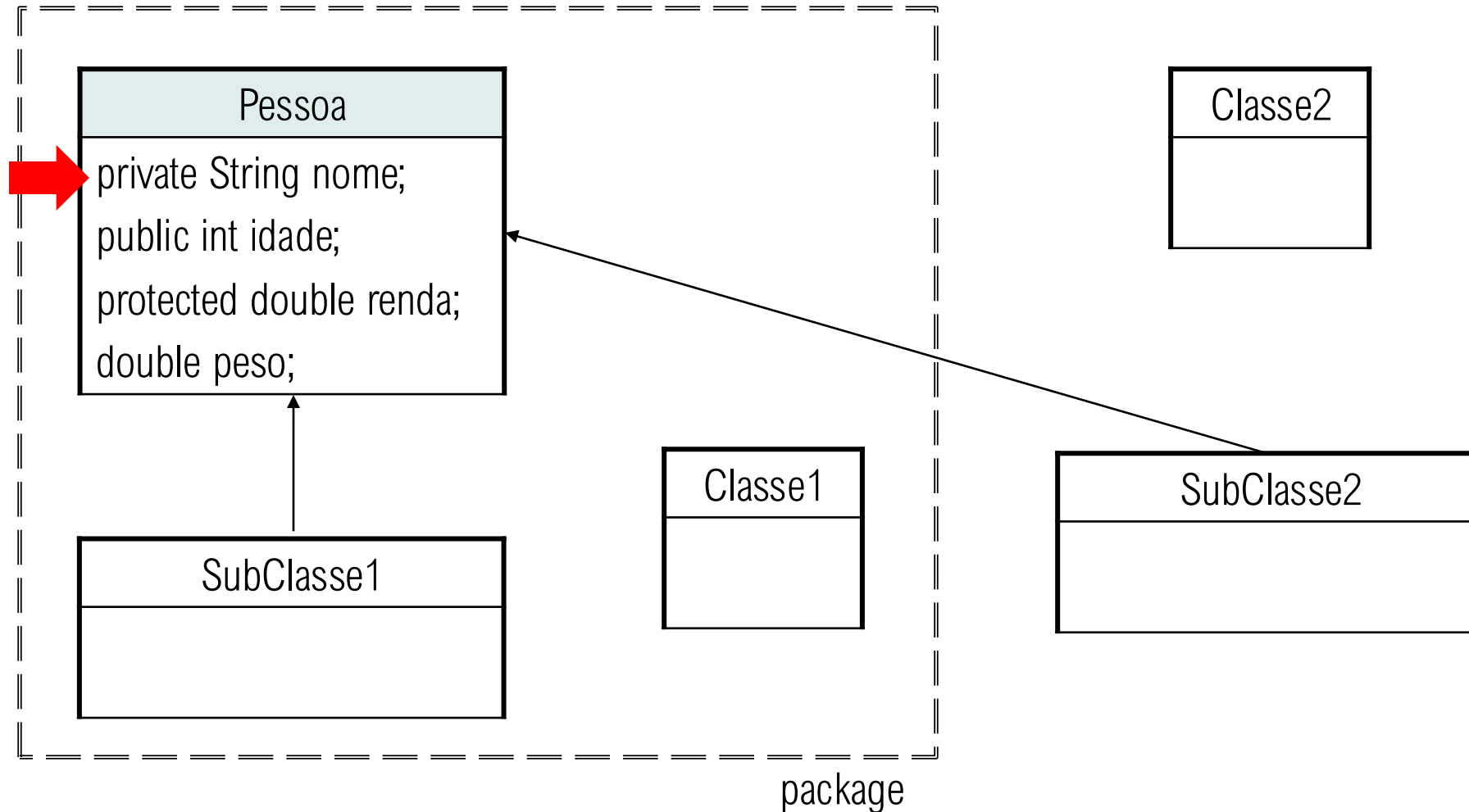
Modificadores de acesso

Visibilidade de variáveis e métodos com o modificador **default**.



Modificadores de acesso

Visibilidade de variáveis e métodos com o modificador **private**.



Exemplo

Pessoa
- nome: String
+ idade: int
renda: double
imprimeDados(): void

Legenda:

(-) indica **private**

(#) indica **protected**

(+) indica **public**

```
import javax.swing.JOptionPane;

public class Pessoa {
    //Atributos
    private String nome;
    public int idade;
    protected double renda;

    //métodos
    public void imprimeDados() {
        JOptionPane.showMessageDialog(null, "Nome: " + nome
            + "\nIdade: " + idade
            + "\nRenda: " + renda);
    }
}
```

Observação: a ausência de sinal na frente do atributo indica que ele é **default**

Exemplo

```
public class TestePessoa {  
    public static void main(String[] args) {  
        Pessoa p1 = new Pessoa();  
  
        p1.nome = "Camila"; //erro de acesso  
        p1.idade = 30; //acesso bem sucedido  
        p1.renda = 1700.55; // acesso bem sucedido  
  
        p1.imprimeDados(); // acesso bem sucedido  
    }  
}
```



Métodos de acesso

- ✓ Servem como métodos de leitura/escrita aos atributos de classes
- ✓ Um método de leitura para um atributo deve ser chamado de **getXxx** (onde Xxx é o nome do atributo). Este método não recebe nada como parâmetro, e retorna o mesmo tipo do atributo.
- ✓ Já um método de gravação deve ser chamado **setXxx**, não retorna nada (geralmente), e recebe como parâmetro o valor que deve ser armazenado no atributo.



Exemplo

Pessoa
- nome: String - idade: int - renda: double
Pessoa() Pessoa(n: String, i: int, r: double) getNome(): String getIdade(): int getRenda(): double setNome(String n): void setIdade(int i): void setRenda(double r): void imprimeDados(): void

```
...
//Métodos de acesso
public String getNome() {
    return nome;
}
public int getIdade() {
    return idade;
}
public double getRenda() {
    return renda;
}
public void setNome(String n) {
    nome = n;
}
public void setIdade(int i) {
    idade = i;
}
public void setRenda(double r) {
    renda = r;
}
...
```

Exercícios



Exercícios

1 - Crie uma classe de nome **Torneio** conforme diagrama e criar classe com void main para instanciar objetos:

Torneio
- nome: string - idade: int
Torneio(nome: string, idade: int) getNome(): string getIdade(): int setNome(n: string): void setIdade(i: int): void verificaCategoria(): string imprimeDados(): void

- a) Crie os sets e gets para cada um dos atributos;
- b) Crie um método imprimirDados que imprime o estado do objeto inclusive sua categoria;
- c) O método verificarCategoria que deverá retornar qual a categoria do atleta baseado na tabela abaixo:

Categoria	Idade
Infantil	5 a 7
Juvenil	8 a 10
Adolescente	11 a 15
Adulto	16 a 30
Sênior	Acima de 30



That's all Folks!